

Jarvis X

Master Plan v3 — with Quantum AI Feasibility Study

Date	3 September 2026
Supersedes	MASTER_PLAN_UPDATED.md (2026-08-12), PLAN.md
Evidence base	AS_BUILT.md, QUANTUM_FEASIBILITY.md, docs/triage/2026-09-repo-triage.md
Completion	62% under the runbook's phase weights
Quantum verdict	Build as a gated research track; keep out of the request path
Reproduce	python3 -m quantum.benchmark

Every number in this document was measured, not estimated. The measuring code is committed in quantum/ so any claim here can be re-run and disputed. Where a figure rests on judgment rather than an instrument, it says so.

Prepared for Ahmed Yidris

1. Verdict first

Seven questions, seven measured answers. Section references point into QUANTUM_FEASIBILITY.md.

Question	Answer	Evidence
Can a quantum simulator run locally with no API?	Yes, comfortably	8 qubits in 9 ms, ~0 MB
How many qubits fit in a request path?	About 16	16q = 32 ms; 24q = 8.4 s
Does the 'LLM -> circuit -> tone' design work?	No. It computes $\cos(\theta)$	matches closed form to 1.45×10^{-16}
Does a real trainable VQC beat what ships today?	No. 0.273 vs 0.500	head-to-head, section 4
Is the blocker qubits, features, or data?	Data. 64 queries	0.595 train / 0.273 test
Should it enter the request path now?	No. 889x slower, less accurate	head-to-head, section 4
Is it worth building anyway?	Yes, as a gated track	section 6

2. Hardware — settled from the repo's own record

The runbook called this open across four candidate machines. The repo answers it:

MASTER_PLAN_UPDATED.md : 45, sourced from WEEK_1_COMPLETE.md, records a verified profile.

Property	Value	Source
Machine	Asus Chromebook	WEEK_1_COMPLETE.md
CPU	Intel i5-1135G7, 8 vCPU	same
RAM	14 GB	corroborated by JARVIS X v2.pdf diagnostics
GPU	none	same
OS	Crostini, Debian 12, kernel 6.6.119	same
Ollama	0.32.9, qwen2.5:7b + 3b	same

The 7.7 GB figure described a different machine. 00-repo-spec.md documents a Lenovo Y50-70, not the dev box. That single mismatch generated most of the four-machine confusion. Model sizing should assume **14 GB, no GPU**. This does not overturn the runbook's verdict on local coding models: 7B-14B at long context remains out of reach on CPU, and the RTX 3060 12 GB is still the unlock.

3. Completion: 62%

One number, derived from the runbook's phase weights. **The weights are the runbook's; the per-phase scores are judgment anchored to cited evidence, not measurements** — so 62% is a defensible estimate, not a reading off an instrument.

Phase	Weight	Score	Contrib.	Basis
Planning	15	90%	13.5	CONSTITUTION, 4 decision records, AS_BUILT, this plan, triage: all current
Setup	10	85%	8.5	Toolchain, Ollama + 4 models, Piper/Kokoro voices, Docker healthy, CI green
Build	45	60%	27.0	Verified: path jail, validation 23/23, data layer, scheduler, i18n/a11y, voice routing, model-gateway 47/47, jj CLI, FastAPI app, TTS engine. Stubbed: Electron (3 files), jj status
Test	20	45%	9.0	13/19 JS files pass but 6 unmeasured ; test-guard and test-shell have zero assertions ; Python suites need deps; E2E is one script
Delivery	10	40%	4.0	Docker works; .deb ships an empty /opt/jarvis-x; remote access is git-only; web/ builds
Total	100		62.0%	

Why not 91%

That figure was scripts/status.sh's 22/24 milestones — and **17 of its 24 checks are bare file-existence tests**. One of them passed happily while code/shell.js held the jail escape fixed in PR #2. It measured file presence, not function.

The four historical figures were measuring four different things:

Figure	What it actually measured
91% (22/24)	status.sh milestones, 17 of which are file-existence checks
49/49	status.sh checks . The doc calling them 'unit tests' is wrong
11/11	the JS suite when it had 11 test files . It now has 19
47/47	packages/model-gateway's own suite only

Biggest uncertainty: Build and Test. Six subsystems (Ollama vision, three voice paths, Kokoro, live data) are *unmeasured* rather than known-broken. If they pass on the real machine, Test rises materially and the total lands nearer 70%.

4. Quantum AI feasibility

4.1 The hardware envelope

Each qubit doubles the state vector, so cost is exponential and the wall arrives abruptly. Between 20 and 24 qubits, latency grows 42x and memory 16x. No configuration or threading trick changes the shape of this curve — it is what simulating 2^n amplitudes costs.

Qubits	State vector	Latency	Peak memory	Usable where
2	4	3.9 ms	~0 MB	anywhere
4	16	5.7 ms	~0 MB	anywhere
8	256	9.3 ms	~0 MB	request path
12	4,096	14.4 ms	0.2 MB	request path
16	65,536	31.9 ms	3.2 MB	request path, at the edge
18	262,144	69.7 ms	12.6 MB	background only
20	1,048,576	199.6 ms	50.4 MB	background only
22	4,194,304	1,123 ms	201 MB	batch / offline
24	16,777,216	8,355 ms	805 MB	unusable

Measured in a cloud container (4 x Xeon @ 2.10 GHz, 15 GB), **not on the Chromebook**. The *shape* of the curve transfers — it is set by 2^n , not by the CPU — but absolute milliseconds do not. Expect the Chromebook to be slower, which *tightens* the 16-qubit ceiling rather than loosening it.

4.2 Why the proposed design does not work

The circulating design has an LLM emit two numbers, feeds them as rotation angles into a 2-qubit circuit, and thresholds the measurement to pick a response tone. The circuit has **no trainable parameters**, so it has a closed form:

urgency	complexity	qubit 0	cos(u)	qubit 1	cos(u)cos(c)
0.500	1.000	0.877583	0.877583	0.474160	0.474160
1.000	2.000	0.540302	0.540302	-0.224845	-0.224845
2.000	0.300	-0.416147	-0.416147	-0.397560	-0.397560
3.140	3.140	-0.999999	-0.999999	0.999997	0.999997

Maximum deviation: 1.45×10^{-16} — floating-point noise. It also returned **one distinct result across 200 identical calls**: fully deterministic, no sampling, no superposition exploited, nothing an if statement could not do more cheaply. This finding is pinned as an executable test (test_naive_circuit_is_just_cosine) so the pattern cannot quietly return.

4.3 The feature-budget trap, and the way out

AngleEmbedding spends **one qubit per feature**, capping you at ~16 features in a request path — hopeless for text, where the seed dataset alone has a 142-word vocabulary. At 8 dimensions, **134 of 142 vocabulary words collide**, so that path does not merely underperform: it destroys the input.

Feature dimensions	8	16	32	64	128	256	512
Classical test accuracy	0.273	0.227	0.273	0.364	0.409	0.364	0.318
Hash collisions (142 words)	134	126	111	88	58	32	19

AmplitudeEmbedding answers this: it packs 2^n features into n qubits, so 8 qubits carry **256** features rather than 8. The quantum feature budget is therefore *not* the blocker — which is what makes the next result meaningful.

4.4 The head-to-head

Task: route a query to one of four data-provider families. The baseline is `resolveQuery()` from `code/agent-data-integration.js` — **the code that ships today**. Both learned models consume identical features. 64 queries, 42 train / 22 test. Accuracy is split by query kind because the aggregate hides the story.

Model	Acc.	on kw	on para	Train	Inference	Params
keyword matcher (ships today)	0.500	1.000	0.000	--	0.005 ms	0
classical softmax (256 feats)	0.364	0.364	0.364	0.04 s	0.012 ms	1,028
quantum VQC (8q/2L, amplitude)	0.273	0.273	0.273	142 s	10.3 ms	48

The quantum classifier loses on every axis at once: less accurate than a zero-parameter keyword matcher, **889x slower** per inference than classical logistic regression, and **3,297x slower** to train.

It was steel-manned before that conclusion. It got amplitude embedding (256 features, not 8), backprop gradients instead of parameter-shift (~100x cheaper), and entangling layers with genuinely trainable weights. It still lost.

What the split reveals

- The keyword matcher is **perfect on keyword queries and scores zero on paraphrases** — it returned null on **11 of 11**. Its weakness is real and precisely located.
- Both learned models score 0.27-0.36 *uniformly* across both kinds. They have not learned the task; they sit near the 0.25 chance line.
- The VQC reached **0.595 train / 0.273 test** — textbook overfitting on 42 samples.

So the binding constraint is training data, not quantum versus classical. With 64 hand-written queries, no classifier of any kind beats the keyword matcher. Buying better hardware or adding qubits would not change this result.

5. What 'best local quantum AI model' can mean

Three separate goals get conflated under that phrase, and they have different verdicts.

Interpretation	Verdict
Run QML locally with no API	Done. PennyLane simulates on CPU, offline, free. Section 4.1 is the envelope.
A quantum model that improves Jarvis today	Not supported by the data (section 4.4). Revisit when the dataset is 100x bigger.
A 'quantum LLM' replacing Ollama	Not physically available. A 1B-parameter model needs $\sim 10^9$ trained parameters; the largest circuits simulable here hold ~ 48 . No hardware, quantum or classical, runs a transformer as a quantum circuit today.

On the suggested model

NeuroEquality/neuralquantum-coder **does exist** on Ollama. But its published description is a generic coding assistant — 'clear, actionable, technically accurate solutions... performance, security, and best practices' — with no published parameter count or base model, reading like a system-prompt wrapper. Nothing found supports the claim that it reasons about quantum structures; 'NeuralQuantum' is the vendor's brand. **Treat as unverified:** the model card could not be read directly, as egress to ollama.com and huggingface.co is blocked from the build environment.

6. Recommendation: build it, gate it, let data promote it

Not 'do not build this'. The module is committed and working. But it earns the request path by measurement, not by intent.

Shipped now

- quantum/ with a real trainable VQC, both baselines, and the benchmark harness
- 31 correctness checks, no accuracy assertions
- PennyLane isolated to quantum/requirements.txt; nothing on the serving path imports it
- The $\cos(\theta)$ finding pinned as an executable regression test

Promotion gate — all four, or it stays out

#	Gate
1	Dataset of 500 or more real queries from logs/queue.jsonl, not hand-written
2	VQC test accuracy beats the keyword matcher on the same split
3	VQC beats classical softmax on identical features
4	p95 inference under 50 ms on Ahmed's actual hardware

Gate 3 is the one that matters. If quantum and classical tie, the answer is classical: same accuracy at 1/889th the cost. A quantum result without a classical control is unfalsifiable — you cannot tell whether the circuit helped or the features were simply learnable.

Do not pursue: circuits above 16 qubits for anything user-facing (4.1), a parameterless circuit in a decision path (4.2), or AngleEmbedding for text (4.3).

7. Where the plan meets the hard constraints

Conflict	Status
Paper trading	Ruled 2026-09-04: delete. code/paper-trading.js and config/trading.json removed; CONSTITUTION.md section IV now forbids trading of any kind, real or simulated, rather than permitting a testnet carve-out. Git history retains the code.
Remote access via 0.0.0.0	Rejected. Every binding in the repo is deliberately 127.0.0.1, port 8000 is already held by the live deployment, and the runbook forbids exposure. Git stays the sync layer.
Local coding models replacing Claude Code	Unchanged: not viable on CPU. Gemini CLI free tier is the out-of-limit answer.
JX_NET gating	Ruled 2026-09-04: gate it. Implemented in code/test-net.js; the network test skips unless JX_NET=1, and jest-runner.js counts skips in their own column so a skip can never read as a pass.
Quantum in the request path	Rejected on measurement: 889x slower and less accurate than the shipped keyword matcher. Kept as a gated research track.

8. Re-sequenced work, in 1-3 hour tasks

Income work first. Per Ahmed's own operating rules: if Outlier/Upwork hours are not done this week, they precede everything below.

Tier 1 — buy real information (do these first)

#	Task	Hrs	Why it is first	Done when
1	Run the JS suite on the dev machine	0.5	Converts 6 unmeasured subsystems into knowns. Highest information per minute in the plan.	Output pasted; AS_BUILT updated
2	Confirm hardware with the Session 1 script	0.25	Closes section 2 empirically	~/PROFILE.md exists
3	Rule on paper trading and JX_NET	0.25	Two one-line answers unblock section 7	Rulings recorded
4	Give test-guard and test-shell real assertions	1.5	Both have zero assertions and pass unconditionally. test-shell is the file that would have caught the jail escape.	Both assert; both fail if reverted

Tier 2 — close the real gaps

#	Task	Hrs	Why	Done when
5	Make the .deb ship app.py	2	Currently installs an empty /opt/jarvis-x; the launcher cannot work	dpkg -i then launch succeeds
6	Decide JX_NET: gate or make offline-capable	1.5	Makes the suite deterministic offline	19/19 or an explicit skip contract
7	Real jj status	2	Prints a hardcoded ready message today; Session 6 wants live quota	Shows per-provider quota and health
8	Per-provider quota counters in SQLite	3	Session 6 proper: free-tier ceilings with 429 fallback	Counters persist; fallback verified
9	Install Piper voices / Ollama where task 1 showed gaps	1-2	Turns unmeasured into passing	Formerly-failing tests pass

Tier 3 — extend

#	Task	Hrs	Why	Done when
10	Build the 500-query dataset from logs/queue.jsonl	2	Quantum gate 1 — and it improves the classical router regardless	quantum/data/ has 500+ real queries
11	Re-run the benchmark on the real dataset	1	Lets gates 2-4 decide on evidence	Results appended to the study
12	Electron shell to a launchable app	3	3 files today	Window opens, talks to the API
13	E2E flow test	2.5	One script today	Ask -> answer -> audio, asserted

Explicitly not scheduled: local 7B+ coding models, LAN or internet exposure, a quantum model in the request path, or quantum circuits above 16 qubits for anything user-facing.

9. Known limits of this study

Stated plainly, so nothing here is over-read.

- **The dataset is 64 hand-written queries.** It is a seed, not a benchmark. A larger real-traffic dataset could change section 4.4 — which is exactly what gate 1 tests.
- **Hardware numbers come from a cloud container,** not the target machine (section 4.1).
- **One circuit architecture tested** (StronglyEntanglingLayers, 1-2 layers, 8 qubits). Other ansatze exist; none was tried.
- **One task tested** (4-way intent routing). QAOA-style optimization for the scheduler was *not* measured — though section 4.1 caps it at ~16 items, a size classical exact solving handles trivially.
- **Simulation only.** No real quantum hardware, which would need the cloud APIs this design explicitly avoids.
- **Not measured:** accuracy at the 11-key granularity resolveQuery actually returns; only the 4-family collapse was tested.
- **The 62% completion figure blends measurement with judgment** (section 3). The phase weights are the runbook's; the per-phase scores are reasoned from cited evidence, not read off an instrument.

What would change this plan

- **Task 1's output.** If the 6 unmeasured subsystems pass, completion moves to ~70% and Tier 2 shrinks. If they fail on the real machine too, they become Tier 1 bugs.
- **A ruling against paper trading.** Deletes two files and needs a CONSTITUTION amendment.
- **The RTX 3060 12 GB.** Unlocks local 14B coding models *and* lifts the quantum ceiling from 16 to 24+ qubits — the one purchase that changes two constraints at once.
- **Task 10's dataset.** The single input most likely to overturn the head-to-head result.

Method follows AS_BUILT.md: every claim cites a command, and anything unverified is labelled unverified rather than estimated. Superseded plans keep their history — MASTER_PLAN_UPDATED.md and PLAN.md are archived, not deleted.